LoGO: Combining Local Deltas and Global Offsets for Data Prefetching

CS683 Project
Checkpoint 1

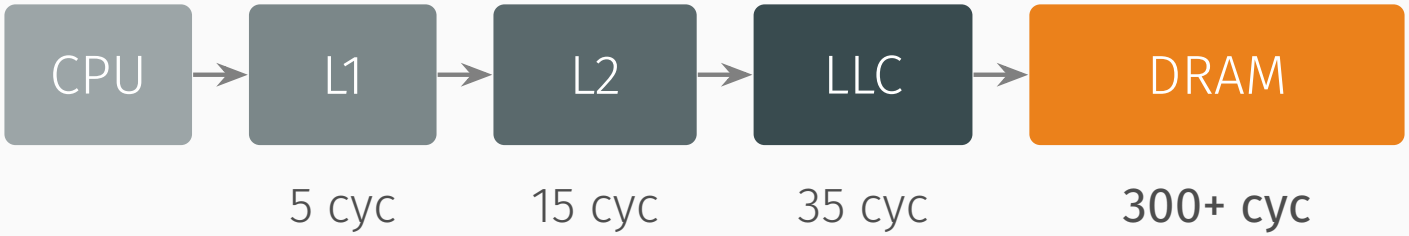
Mradul Sonkar

23B0980

Instructor: Biswabandan Panda
Department of Computer Science and Engineering, IIT Bombay

Why Data Prefetching?

Memory is far from the CPU



Prefetcher: guess the next address, fetch it early

1.74×

faster with Berti

(Trace - **fotonik3d**)

0.065

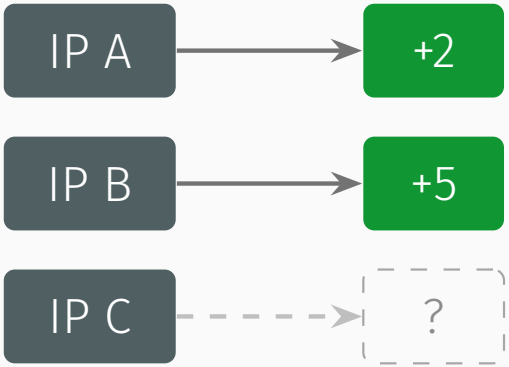
instructions/cycle without one

(Trace - **pr**, waiting on memory)

Two Ways to Guess

Local

one delta *per instruction* (Berti)

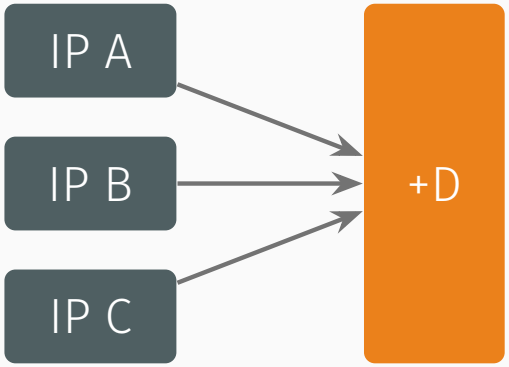


✗ silent when unsure

✓ 93% accurate

Global

one offset *for everything* (BOP)



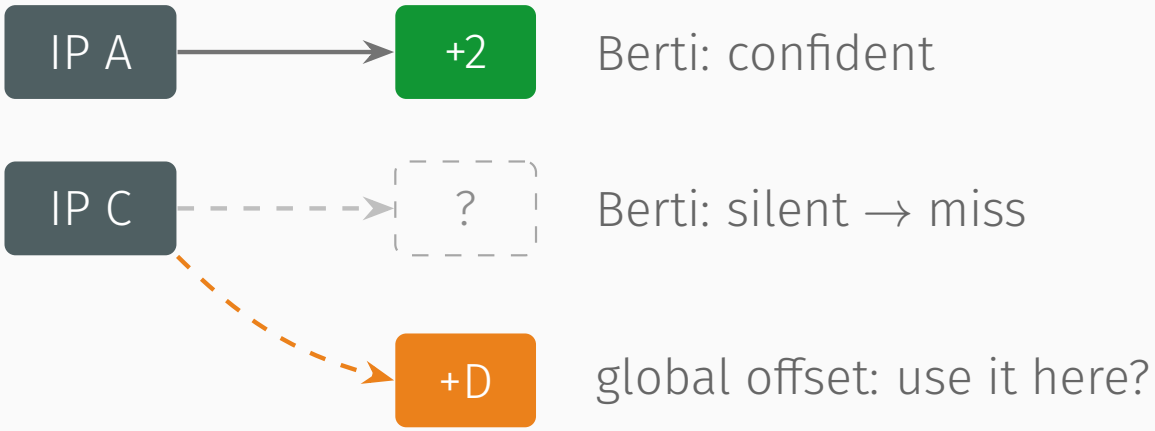
✓ always guesses

✗ 48% accurate

∗ Accuracy = of all lines prefetched, the share the program actually used.

∗ Average over 9 program traces: 6 standard CPU benchmarks (e.g. mcf, lbm) and 3 graph algorithms.

Problem Statement



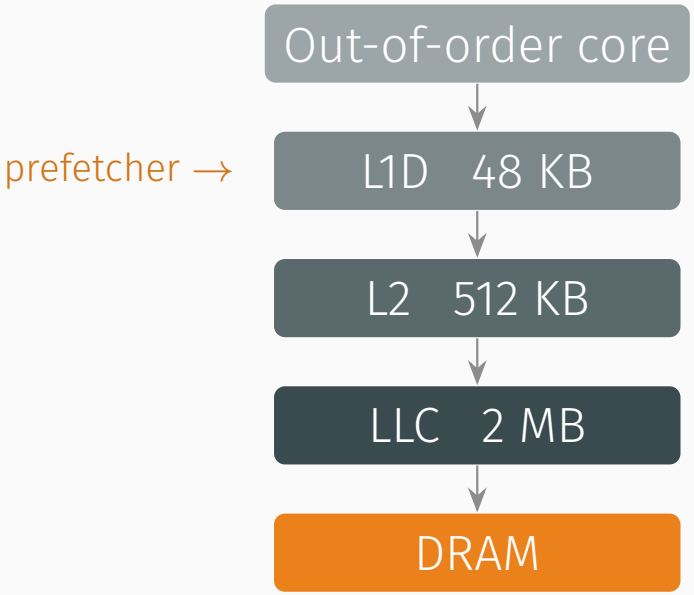
Can a global offset fill Berti's silence without losing accuracy?

We ask a few questions here:

1. How many misses happen while Berti is silent?
2. How many could a global offset catch?
3. How accurate would those guesses be?

Experimental Setup

Machine



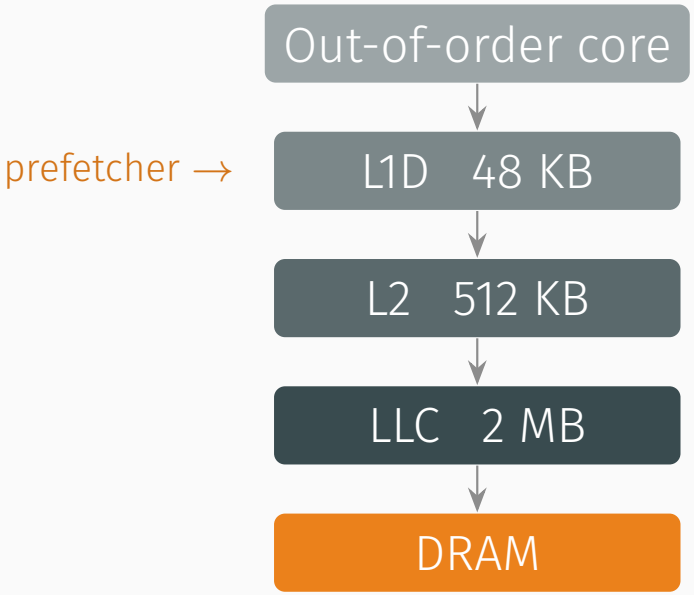
ChampSim, 1 core

50M + 200M instructions

(warm-up + measured)

Experimental Setup

Machine



ChampSim, 1 core
50M + 200M
instructions

(warm-up + measured)

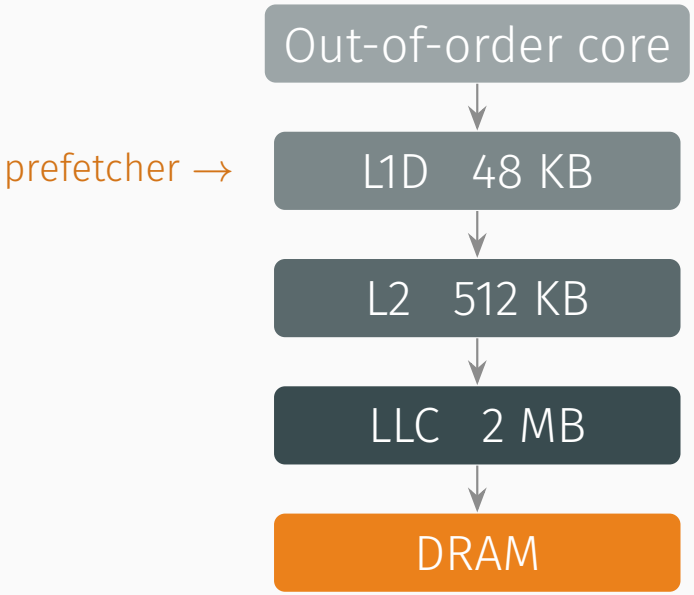
Traces (9 programs)

Regular	bwaves	lbm	fotonik3d
Irregular	mcf	omnetpp	xalancbmk
Graph	BFS	PageRank	Conn. Comp.

6 standard CPU benchmarks + 3 graph algorithms

Experimental Setup

Machine



ChampSim, 1 core
50M + 200M
instructions

(warm-up + measured)

Traces (9 programs)

Regular	bwaves	lbm	fotonik3d
Irregular	mcf	omnetpp	xalancbmk
Graph	BFS	PageRank	Conn. Comp.

6 standard CPU benchmarks + 3 graph algorithms

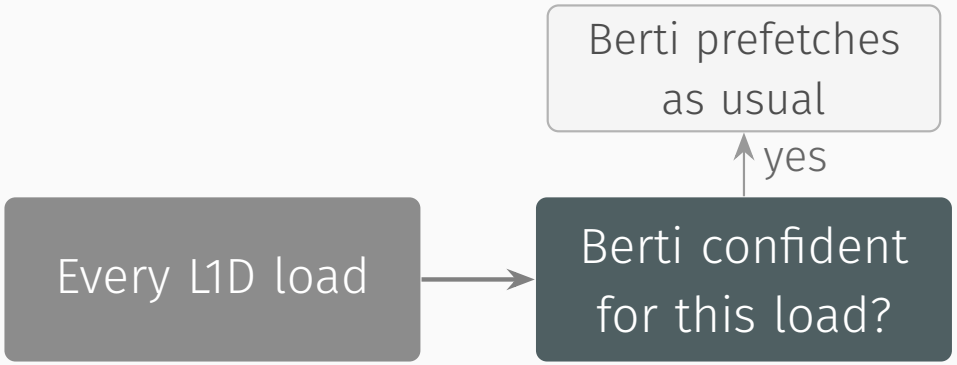
Prefetchers (at L1D)

No prefetching | IP-stride

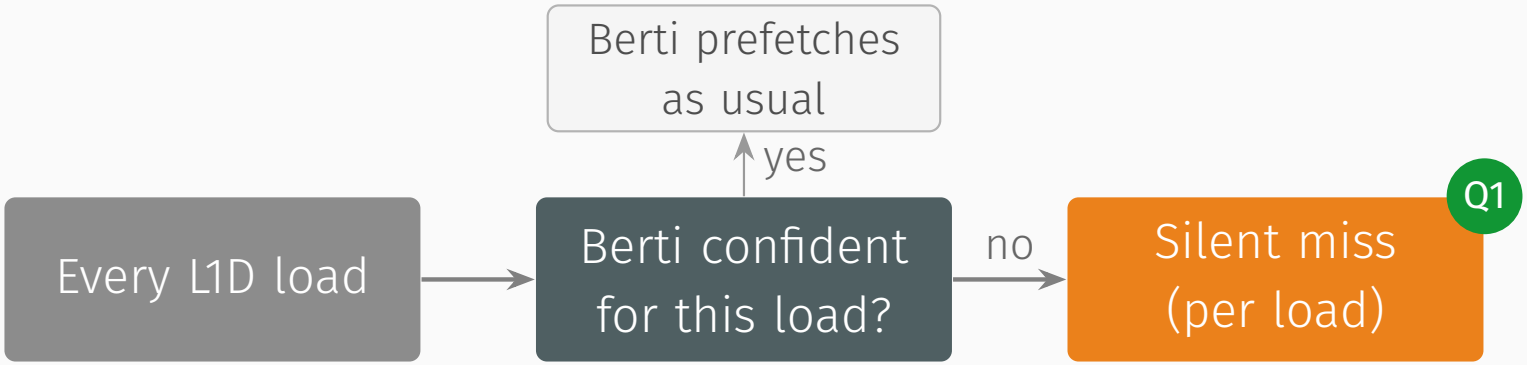
BOP* | Berti

∗ implemented from the BOP paper (HPCA 2016)

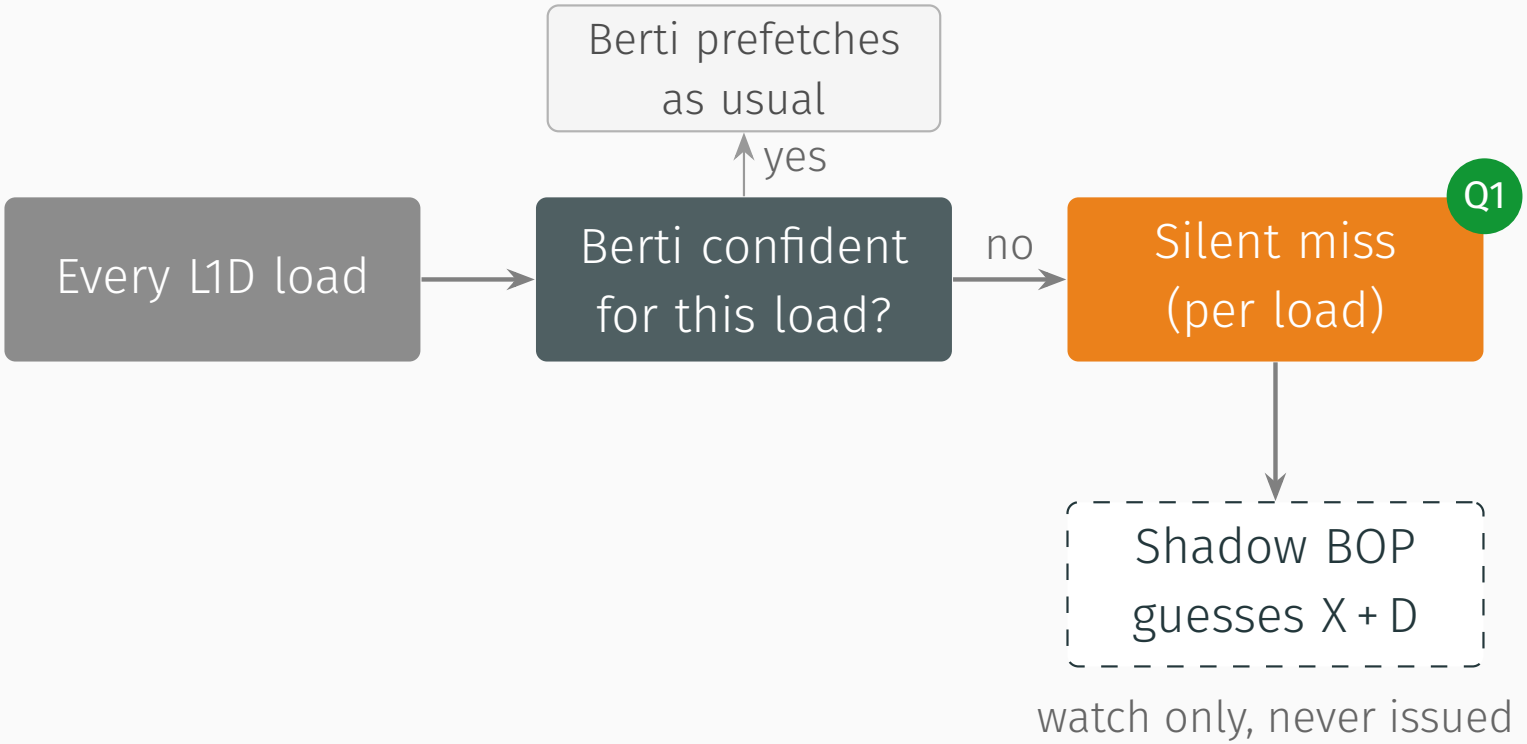
Measuring Berti's Silence



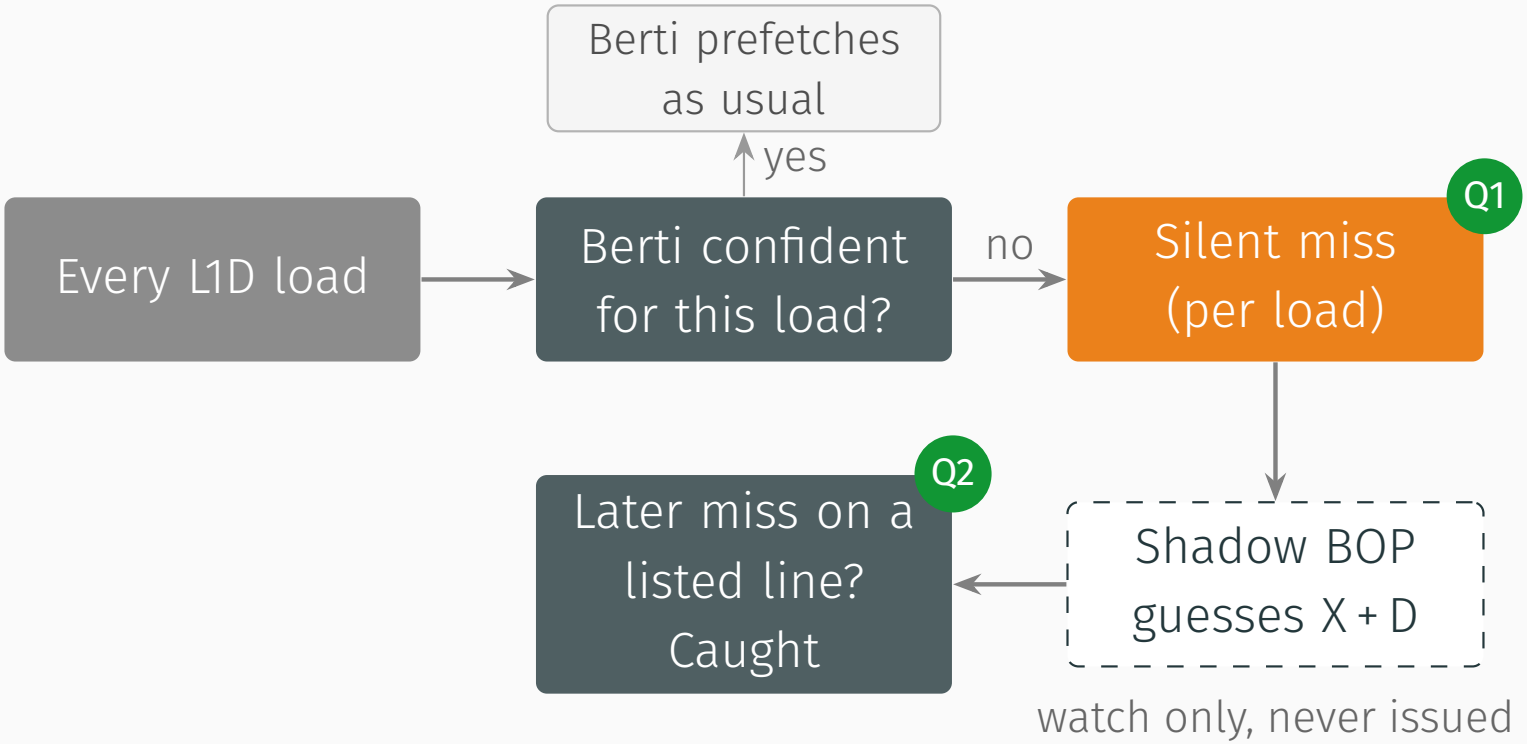
Measuring Berti's Silence



Measuring Berti's Silence



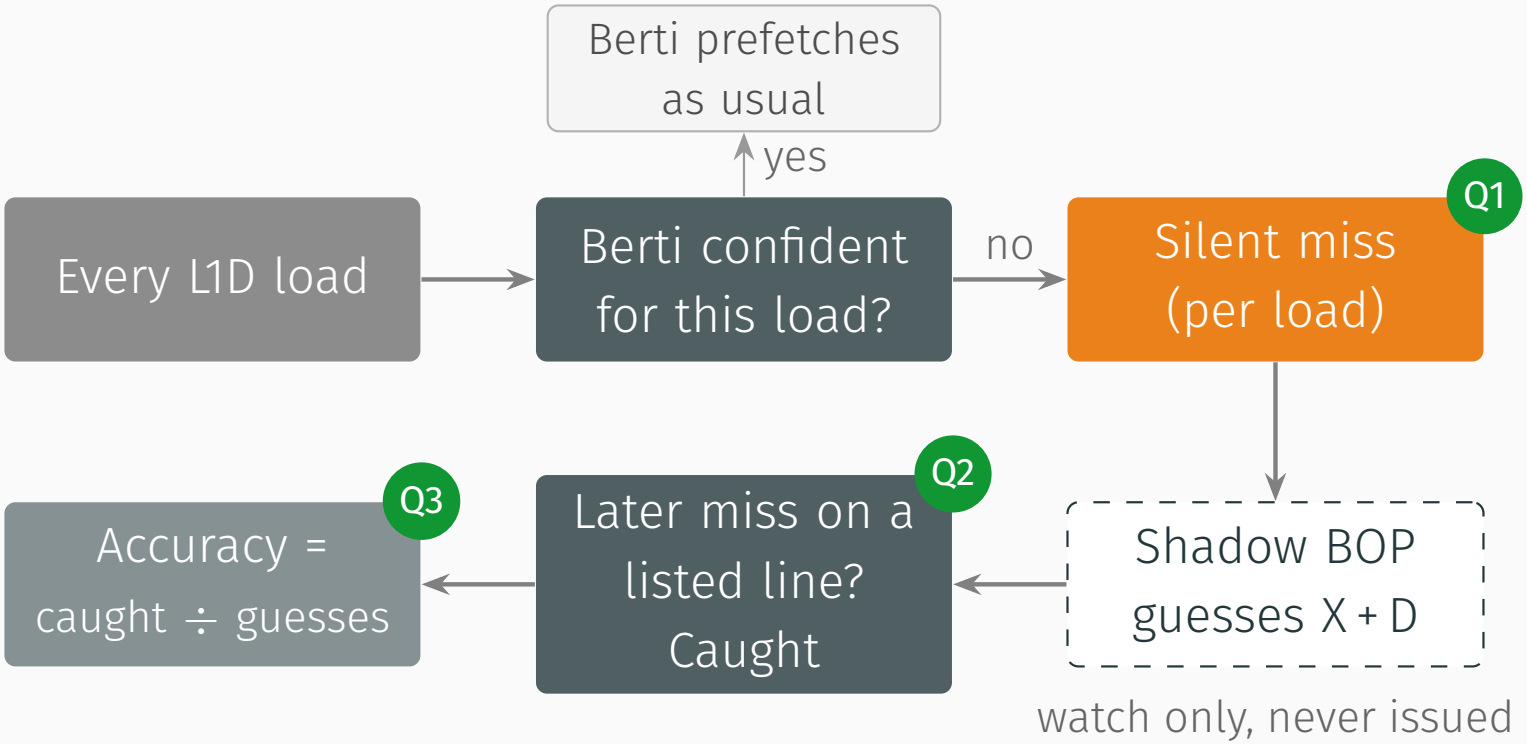
Measuring Berti's Silence



How do we know a guess would have helped?

1. Store X + D in a *would-prefetch list* (768 lines, L1D size)
2. A later miss on a listed line would have hit → caught

Measuring Berti's Silence

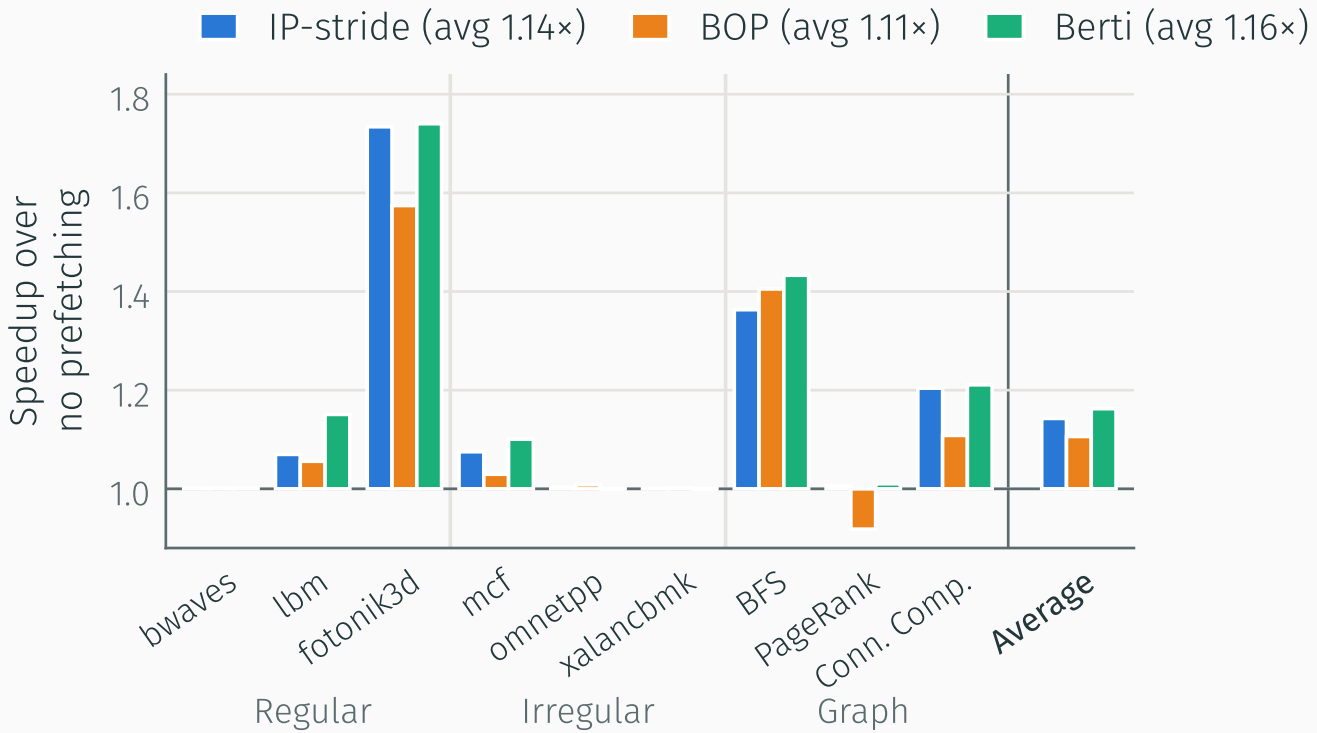


How do we know a guess would have helped?

1. Store X + D in a *would-prefetch list* (768 lines, L1D size)
2. A later miss on a listed line would have hit → caught

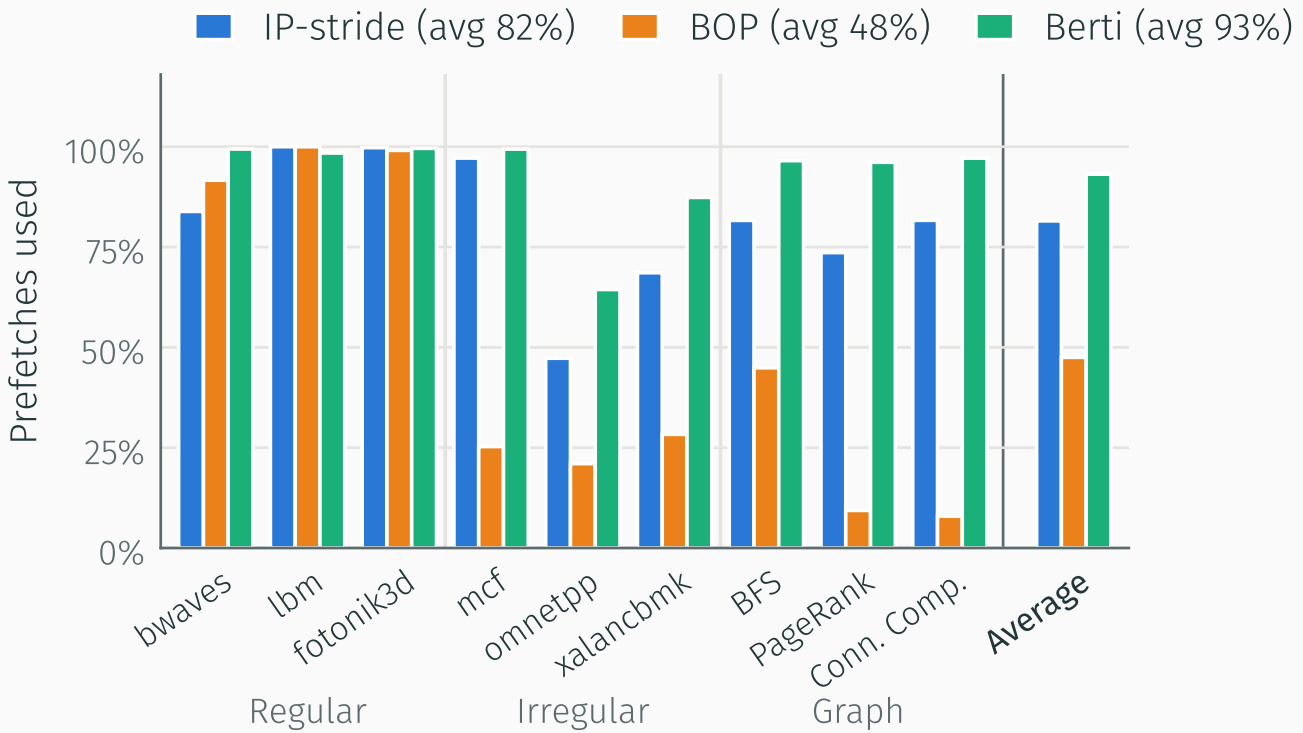
Baselines: Speedup

Berti is the strongest baseline: 1.16× on average



Baselines: Accuracy

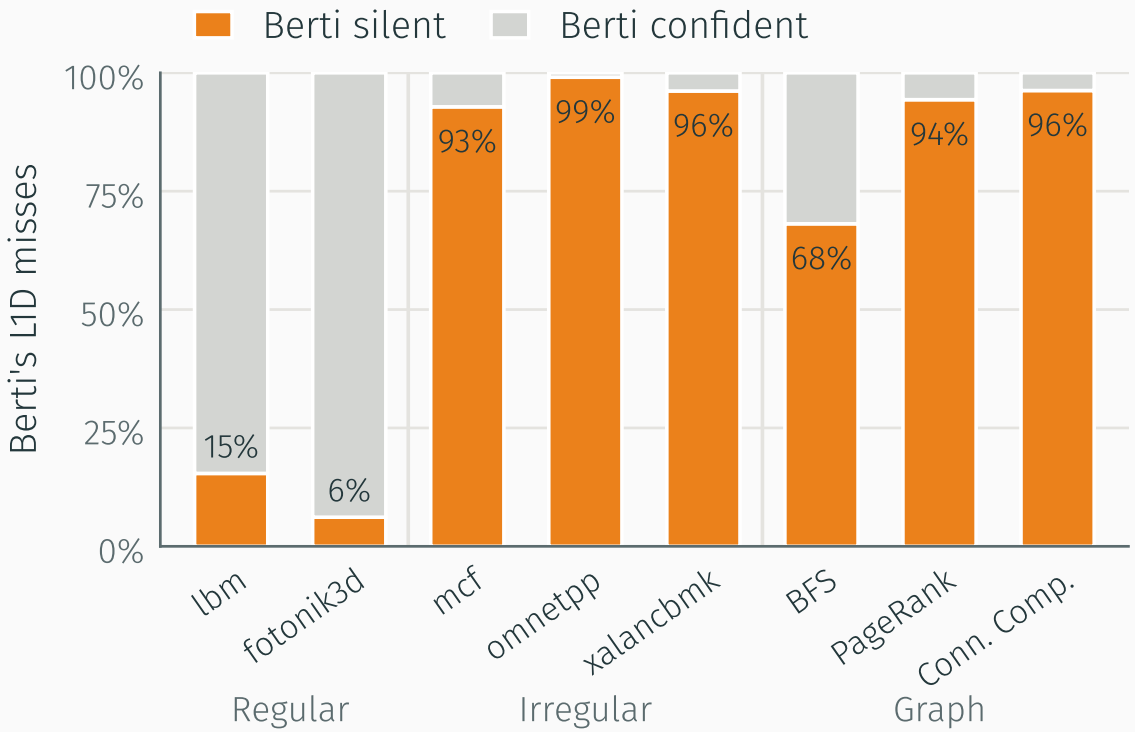
Berti stays accurate by staying silent;
BOP's guesses collapse on irregular and graph code



Prefetches used = of all lines prefetched, the share the program actually used.

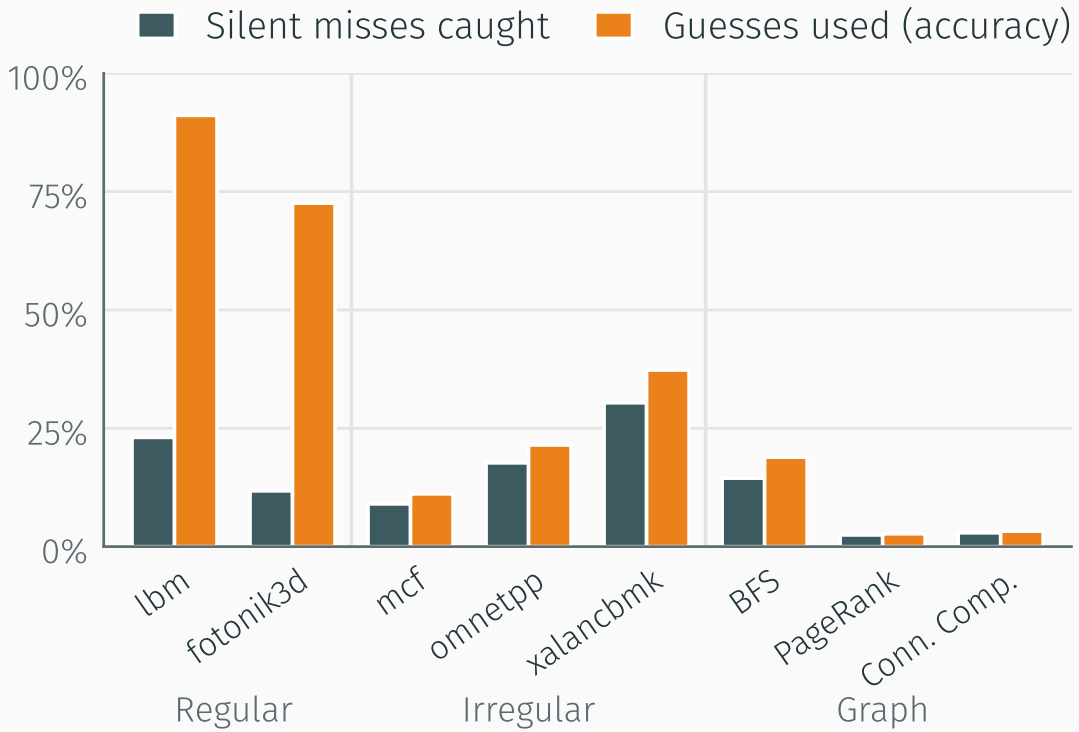
Q1: How Big Is the Gap?

On irregular and graph code, **68–99%** of Berti's misses happen while Berti is silent



Q2 + Q3: Can a Global Offset Fill It?

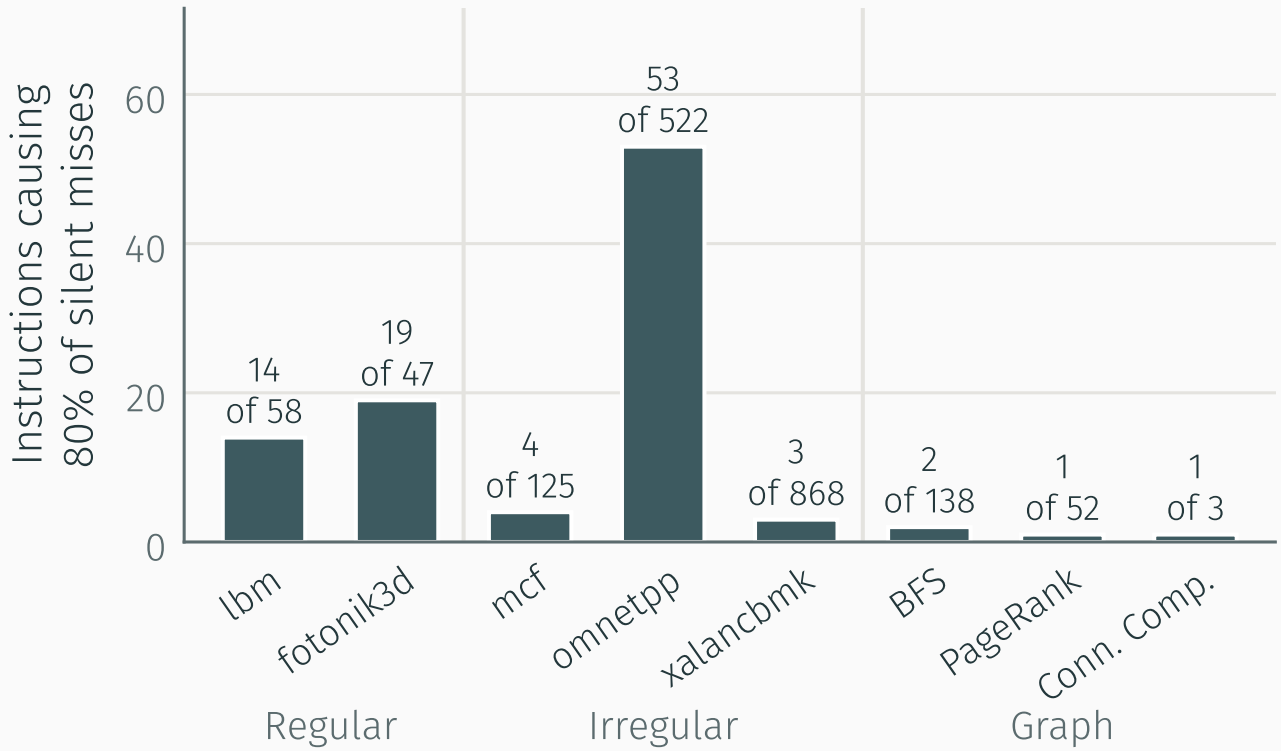
Mostly no: it catches ≤30% of silent misses, and ≤37% of its guesses are right



Regular code: accurate guesses, but little left to catch

Who Causes the Gap?

A few hot instructions, not cold ones



Instructions with $<$1,000 loads cause $\leq$0.2% of silent misses on irregular and graph code

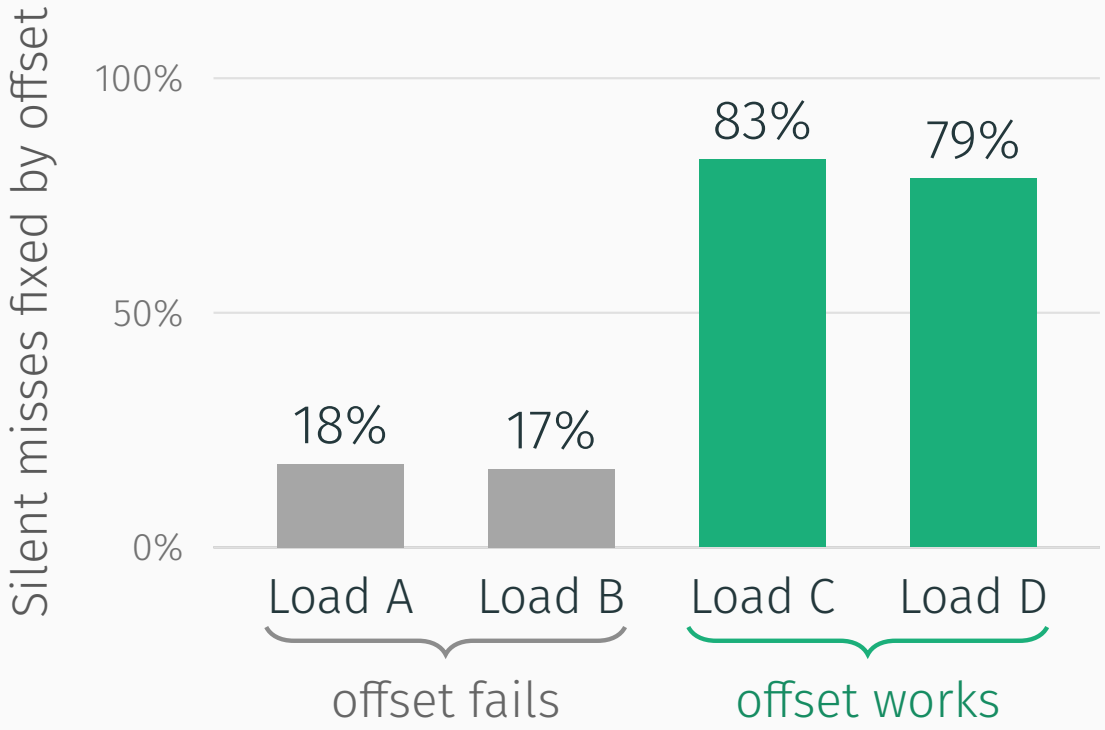
Key Takeaways

93%	of Berti's prefetches are right
	↓ *because it stays silent when unsure*
68–99%	of its misses happen while it is silent
	↓ *a global offset fixes only*
$\leq$30%	of these silent misses
	↓ *but*
1–4	instructions cause 80% of them

$\Rightarrow$ Fix silent misses per instruction, not globally

Answer: It Depends on the Instruction

xalancbmk: the 4 loads with the most silent misses



Same program, same offset
turn it on only for loads where it works

Plan for Checkpoint 2

1. **Measure:** for each load, how often the offset's guess is right

2. **Build LoGO:** a small table (8 loads, ≈30 bytes)
   that turns the offset on only where it is right

3. **Compare** LoGO with Berti and BOP: speed and accuracy

Thank You!